

Barrier Option Pricing Report

Abstract:

This report contains the prices of barrier options obtained via Monte Carlo simulation, where the barrier options have the parameters:

Specifications: Up-and-out European Call

Maturity: 6 months

Underlying asset volatility: 30%

No dividends Risk free rate: 2%

Strike: \$105

Current spot: \$100

Case 1: Constant barrier at \$110 effective in the last 3 months

Case 2: Step-up barrier: \$108 for the first 3 months, \$110 in the last 3 months

Case 3: Linearly increasing barrier: From \$105 to \$115 during the 6 months

Case 4/5/6: Constantly discretely sampled barrier at \$110, where case 4 is monthly, case 5 is weekly and case 6 is hourly

Each case with rebates \$0, \$4 and \$8.

The assumptions made for the barrier options are:

1. Rebate is paid at maturity regardless of when the barrier price is hit.
2. A year has 252 trading days, and 1 trading month is 21 days, 1 trading week is 5 days and 1 trading day is 1 day.
3. Pricing of the derivatives are done under risk-neutral measure with the underlying asset following Geometric Brownian Motion with Black-Sholes assumptions.

Methods:

Each unique barrier option is priced with 3 different methods: Simple, antithetic and control variate Monte Carlo. First, 20000 hourly paths (and 20000 anti-paths) of length $252/2=126$ days are generated from the class Path:

```

class Path:
    def __init__(self, price, days, vol, interest, seed=87, antithetic=True):
        self.paths = []
        self.antipaths = []
        self.price = price
        self.days = days
        self.vol = vol
        self.hourly_vol = vol/np.sqrt(252*24)
        self.drift = interest/(252*24) - 0.5*self.hourly_vol**2
        self.antithetic = antithetic
        np.random.seed(seed)

    def add_path(self, path):
        self.paths.append(path)

    def generate_path(self):
        path = [self.price]
        antipath = [self.price]
        for _ in range(self.days*24):
            Z = np.random.normal(0,1)
            step_change = np.exp(self.drift + self.hourly_vol * Z)
            path.append(path[-1] * step_change)

            if self.antithetic:
                antistep_change = np.exp(self.drift + self.hourly_vol * -Z)
                antipath.append(antipath[-1] * antistep_change)

        self.add_path(path)
        if self.antithetic:
            self.antipaths.append(antipath)

```

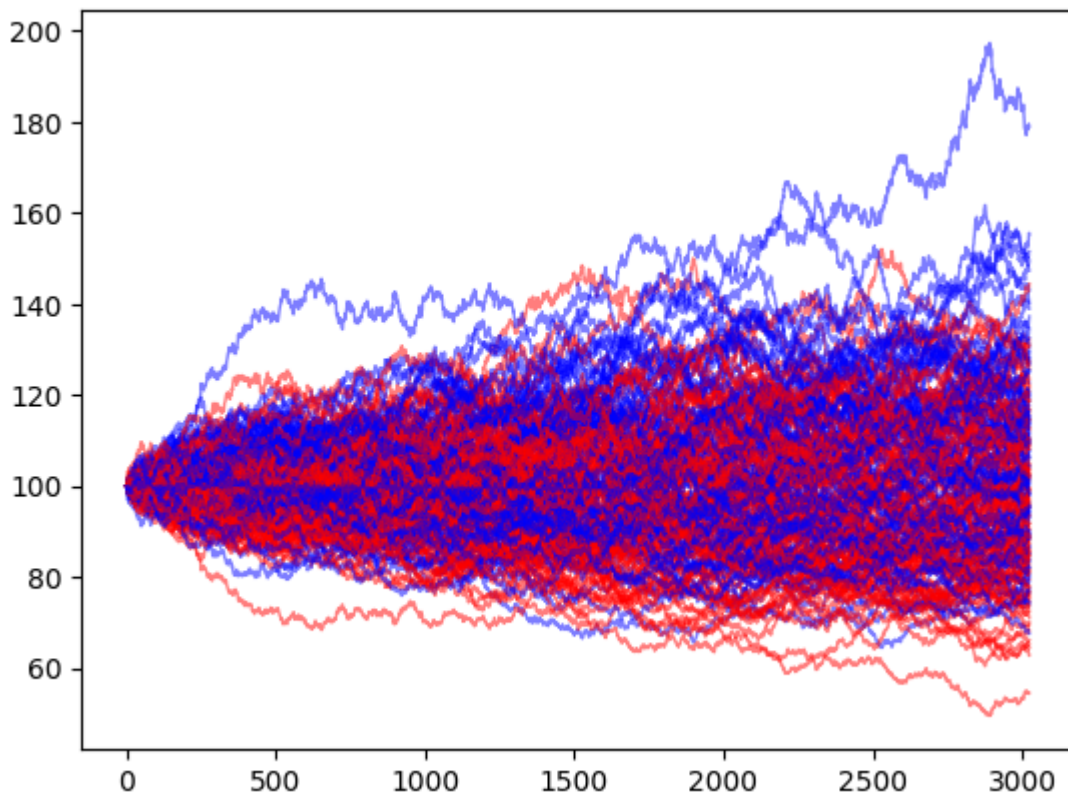
With the parameters:

```

final_path_list = []
paths = Path(price=100, days=126, vol=0.3, interest=0.02)
paths.make_paths(amount=20000)
final_paths = paths.get_paths()
final_antipaths = paths.get_antipaths()

```

This is the first 100 paths (and anti-paths) from the set of paths generated:



And then each barrier option is priced in class Monte:

```
class Monte:
    def __init__(self, paths, antipaths, barrier_type, rebate=0, interval=1):
        self.paths = np.array(paths)
        self.antipaths = antipaths
        self.barrier_type = barrier_type
        self.rebate = rebate
        self.interval = interval
        self.adj = True if interval == 1 else False
        self.discount = np.exp(-0.15 * np.sqrt(self.interval / (252*24)))
```

With barrier parameters:

```
def barrier(self, type, time, cust=110):
    if type == 1:
        barrier_level = 1000000 if time < 63*24 else 110
    elif type == 2:
        barrier_level = 108 if time < 63*24 else 110
    elif type == 3:
        weight = time / (126*24)
        barrier_level = 105*(1-weight) + 115*weight
    elif type == 0:
        barrier_level = cust
    else:
        return ValueError
    if self.adj == True:
        barrier_level *= self.discount
    return barrier_level
```

Where type 1, 2, 3 corresponds to the case 1, 2, 3 of above barrier options.

For simple, antithetic and control variate Monte Carlo, the following functions in the class generate the simulated results:

```
def simple(self, interval=None, print_=True):
    if interval is None:
        interval = self.interval
    payoffs = []
    for path in self.paths:
        for i in range(len(path)):
            if path[i] >= self.barrier(self.barrier_type, i) and (i + 1) % interval == 0:
                payoffs.append(self.rebate*np.exp(0.02*-0.5))
                break
            if i == len(path) - 1:
                payoffs.append(max(path[-1]-105, 0)*np.exp(0.02*-0.5))
    if print_:
        print("Mean payoff:", np.mean(payoffs))
        print("Standard Error:", np.std(payoffs)/np.sqrt(len(payoffs)))
        print("-----")
    return np.array(payoffs)
```

```

def antithetic(self, interval=None):
    if interval is None:
        interval = self.interval
    payoffs_1 = []
    for path in self.paths:
        for i in range(len(path)):
            if path[i] >= self.barrier(self.barrier_type,i) and (i + 1) % interval == 0:
                payoffs_1.append(self.rebate*np.exp(0.02*-0.5))
                break
            if i == len(path) - 1:
                payoffs_1.append(max(path[-1]-105,0)*np.exp(0.02*-0.5))
    payoffs_2 = []
    for path in self.antipaths:
        for i in range(len(path)):
            if path[i] >= self.barrier(self.barrier_type,i) and (i + 1) % interval == 0:
                payoffs_2.append(self.rebate*np.exp(0.02*-0.5))
                break
            if i == len(path) - 1:
                payoffs_2.append(max(path[-1]-105,0)*np.exp(0.02*-0.5))
    print("Corr:",np.corrcoef(payoffs_1,payoffs_2)[0][1])
    payoffs = (np.array(payoffs_1) + np.array(payoffs_2))/2
    print("Mean payoff:",np.mean(payoffs))
    print("Standard Error:",np.std(payoffs)/np.sqrt(len(payoffs)))
    print("-----")
    return payoffs

```

```

def control_variate(self, interval=None):
    if interval is None:
        interval = self.interval
    payoffs = self.simple(interval=interval,print_=False)
    if self.adj == True:
        control_payoffs = self.simple_barrier(cust=110)
    else:
        control_payoffs = self.simple_barrier(cust=110*np.exp(0.15*np.sqrt((interval)/252/24)-0.15*np.sqrt(1/252/24)))
    cov = np.cov(payoffs,control_payoffs)[0][1]
    sample_variance = np.var(control_payoffs)
    if sample_variance == 0:
        sample_variance = 10**-7
    w = cov / sample_variance
    adjusted_payoffs = []
    if self.adj == True:
        adjusted_payoffs = payoffs - w * (control_payoffs - up_and_out_european_option(S=100,K=105,H=110,T=0.5,r=0.02,
                                                                                       q=0.0,sigma=0.3,rebate=self.rebate))
    else:
        adjusted_payoffs = payoffs - w * (control_payoffs - up_and_out_european_option(S=100,K=105,
                                                                                       H=110*np.exp(0.15*np.sqrt(interval/252/24)),
                                                                                       T=0.5,r=0.02,q=0.0,sigma=0.3,rebate=self.rebate))
    print("Corr:",np.corrcoef(payoffs,control_payoffs)[0][1])
    print("Mean payoff:",np.mean(adjusted_payoffs))
    print("Standard Error:",np.std(adjusted_payoffs)/np.sqrt(len(adjusted_payoffs)))
    print("-----")
    return adjusted_payoffs

```

With support functions:

```

def simple_barrier(self, cust=110):
    payoffs = []
    for path in self.paths:
        for i in range(len(path)):
            if path[i] >= self.barrier(0,i,cust=cust):
                payoffs.append(self.rebate*np.exp(0.02*-0.5))
                break
            if i == len(path) - 1:
                payoffs.append(max(path[-1]-105,0)*np.exp(0.02*-0.5))
    return np.array(payoffs)

```

And up_and_out_european_option which is a simple barrier option pricing function.

Note that in some functions, the payoff is discounted and sometimes the barrier level is adjusted based on sampling interval to ensure high accuracy.

Outputs:

Below is the output generated by the python script which is compiled into a table with fair value (expected discounted payoff), standard error of the simulation and correlation. (For antithetic: the correlation of payoffs of the barrier option under each path and its corresponding anti-path. For control variate: the correlation of payoffs of the barrier option and its control asset under each path)

The control selected here is the simple european barrier option with the same parameters, with the barrier level adjusted upwards if the barrier option is discretely sampled.

Case	Rebate	Method	Value	S.E.	Correlation
1	0	Simple	0.0174	0.0014	N/A
1	0	Antithetic	0.0176	0.0010	-0.0074
1	0	Control	0.0175	0.0008	0.8488
1	4	Simple	2.1012	0.0139	N/A
1	4	Antithetic	2.0965	0.0047	-0.7685
1	4	Control	2.1042	0.0084	0.7996
1	8	Simple	4.1851	0.0279	N/A
1	8	Antithetic	4.1753	0.0095	-0.7698
1	8	Control	4.1909	0.0168	0.7987
2	0	Simple	0.0107	0.0011	N/A
2	0	Antithetic	0.0108	0.0008	-0.0043
2	0	Control	0.0108	0.0004	0.9412
2	4	Simple	2.7191	0.0130	N/A
2	4	Antithetic	2.7177	0.0068	-0.4514
2	4	Control	2.7221	0.0057	0.8981
2	8	Simple	5.4274	0.0260	N/A
2	8	Antithetic	5.4246	0.0136	-0.4541
2	8	Control	5.4335	0.0114	0.8979
3	0	Simple	0.0490	0.0035	N/A
3	0	Antithetic	0.0479	0.0024	-0.0096
3	0	Control	0.0470	0.0034	0.2788
3	4	Simple	2.8734	0.0126	N/A
3	4	Antithetic	2.8755	0.0071	-0.3656
3	4	Control	2.8759	0.0081	0.7691
3	8	Simple	5.6978	0.0250	N/A
3	8	Antithetic	5.7031	0.0138	-0.3873
3	8	Control	5.7028	0.0160	0.7673
4	0	Simple	0.0750	0.0034	N/A
4	0	Antithetic	0.0772	0.0024	-0.0250
4	0	Control	0.0752	0.0031	0.4142
4	4	Simple	1.9745	0.0139	N/A
4	4	Antithetic	1.9797	0.0057	-0.6595
4	4	Control	2.0091	0.0059	0.9051

4	8	Simple	3.8740	0.0277	N/A
4	8	Antithetic	3.8823	0.0115	-0.6538
4	8	Control	3.9440	0.0117	0.9056
5	0	Simple	0.0485	0.0031	N/A
5	0	Antithetic	0.0508	0.0022	-0.0131
5	0	Control	0.0520	0.0023	0.6700
5	4	Simple	2.2371	0.0139	N/A
5	4	Antithetic	2.2416	0.0059	-0.6342
5	4	Control	2.2612	0.0046	0.9423
5	8	Simple	4.4257	0.0276	N/A
5	8	Antithetic	4.4324	0.0116	-0.6451
5	8	Control	4.4705	0.0093	0.9414
6	0	Simple	0.0215	0.0017	N/A
6	0	Antithetic	0.0241	0.0013	-0.0084
6	0	Control	0.0227	0.0008	0.8697
6	4	Simple	2.3913	0.0136	N/A
6	4	Antithetic	2.3920	0.0061	-0.6018
6	4	Control	2.4101	0.0033	0.9711
6	8	Simple	4.7611	0.0273	N/A
6	8	Antithetic	4.7599	0.0121	-0.6071
6	8	Control	4.7973	0.0066	0.9705